# Assignment 8: Builder Journal

## What I Built

A tool-using AI agent — not a chatbot — for managing tasks, notes, meetings, and daily/weekly planning, built on the Gemini API. It has 12 tools across task management, notes, meeting-action extraction, and planning; a human-approval gate with an editable-arguments option before any write action; session memory; a SQLite persistence layer; full execution logging; and a pytest suite covering the tools, the approval logic, and the execution limits.

## Most Difficult Technical Problem

Getting multi-step tool-calling to behave reliably in two specific ways: (1) the agent silently created three tasks from a single request without ever pausing for approval, even though Requirement 7 and my own system prompt said multi-task creation must be approved; and (2) the agent kept asking me to convert “due tomorrow” into an ISO date myself instead of just working it out.

## How I Solved It

For the approval gap, I stopped trusting the prompt alone and added a hard, code-level counter in the Agent Controller: it now force-gates the 2nd and every subsequent create_task call within the same turn behind approval, regardless of what the model decides to do. For the date problem, I inject the actual current date into the system instruction on every single call, so the model has the fact instead of guessing — or worse, asking me every time.

## Tool-Calling Errors Observed

Three concrete ones: multiple create_task calls executing without the required approval during early testing; occasional empty “no response produced” messages immediately after a tool executed; and Gemini API quota errors that temporarily prevented the model from responding. These issues were identified through testing and resolved by improving approval validation, response handling, and API error handling.

## Agent Behavior That Surprised Me

That the model has no innate sense of “today.” I assumed an LLM would obviously know what day it is; instead it asked me to do the date conversion myself until I explicitly told it the current date on every call. It was a good reminder that the model only knows what is actually in its context window — nothing more.

## What Failed During Testing

The first live run — “create a task: buy groceries, high priority, due tomorrow” — initially returned a clarification request instead of creating the task. During early development, creating three tasks from one request also bypassed the approval step, violating Requirement 7. After improving the approval logic and prompt handling, these issues were resolved, and the final testing confirmed correct behavior across all required workflows.

## What I Would Redesign

I would flip the default for any new write tool from “no approval unless flagged” to “approval required unless explicitly justified otherwise.” The multi-task gap existed in the first place because the default favored convenience over safety — the opposite of how it should work for anything irreversible.

## What I Learned About Agent Reliability

A prompt instruction is a request, not a guarantee. Every safety rule from Requirement 7 was ultimately enforced in code and verified through testing. I learned that reliable AI agents require deterministic validation, proper approval checks, extensive testing, and clear execution logging rather than relying entirely on the language model’s reasoning.

## Goals for Week 4

Continue improving the agent by expanding evaluation with additional real-world scenarios, refining the system prompt based on testing results, strengthening session memory, improving error recovery, and enhancing the overall user experience. I also plan to further optimize execution performance and explore additional productivity features such as semantic note search and richer productivity analytics.